Master-Seminar

Construction Theory of Information und Software in the Natural World

Aufgabenblatt 1 – Team 1

Physikalisches Basismodell: DLVO, van der Waals und Casimir–Lifshitz

Fokus

Schrittweise Modellierung der effektiven Wechselwirkungen zwischen präbiotischen Protozellen mit R = 500 nm im Bereich L = 2–100 nm. Das Team vergleicht klassische DLVO-Physik inklusive nicht-retardierter van-der-Waals-Anziehung mit einer retardierten Casimir–Lifshitz-Approximation und bereitet robuste Mikrodaten für Team 2 vor.

Leitfrage

Welche Kraft- und Potentialmodelle erzeugen unter präbiotisch plausiblen Bedingungen stabile, persistente oder nur transiente Protozellenkontakte – und in welchem Abstandsbereich kann klassische DLVO-Physik, nicht-retardierte vdW-Physik oder eine retardierte Casimir–Lifshitz-Approximation sinnvoll zur Clusterbildung beitragen?

Feld	Angabe
Team	Team 1 – Physikalisches Basismodell
Zeithorizont	8 Wochen, mit Zwischenstand nach Woche 2
Programmiersprache	Python 3, NumPy, Pandas, Matplotlib, optional NetworkX
Hauptprodukte	Kraft-/Potentialkurven, Brownian-Pair-Simulation, Mehrteilchen-Simulation, Exportdaten für Team 2
Wichtigster Grundsatz	vdW und Casimir–Lifshitz nicht blind addieren: vdW ist der nicht-retardierte Nahbereichsgrenzfall der Lifshitz-Theorie.

1. Ausgangspunkt und wissenschaftliche Rolle von Team 1

Team 1 bildet die physikalische Basis des gesamten Seminars. Alle späteren Teams hängen davon ab, ob die mikroskopische Dynamik sauber, reproduzierbar und physikalisch plausibel modelliert wird. Team 2 benötigt robuste Abstandszonen und Kontaktgraphen. Team 3 benötigt lange Zustandssequenzen. Team 4 benötigt zuverlässige Mikro-/Makro-Vergleiche. Team 5–8 benötigen stabile oder instabile Kontakt-, Task- und Sustainability-Metriken.

Die zentrale Aufgabe ist daher nicht, eine maximale physikalische Vollständigkeit zu erreichen, sondern ein transparentes, schrittweise verfeinerbares Modell zu erstellen, das klar zwischen klassischer Kolloidphysik, nicht-retardierter van-der-Waals-Physik und retardierter Casimir–Lifshitz-Physik unterscheidet.

2. Didaktische Lernziele

Nach Abschluss der Arbeit soll das Team erklären und praktisch demonstrieren können: 1) wie man aus einer physikalischen Theorie ein numerisches Minimalmodell ableitet; 2) warum DLVO, vdW und Casimir–Lifshitz nicht beliebig addiert werden dürfen; 3) wie Debye-Screening die elektrostatische DLVO-Repulsion beeinflusst; 4) warum Salzlösungen die nicht-retardierten und retardierten Fluktuationsbeiträge nicht einfach exponentiell eliminieren; 5) wie Retardierung die Kraft im Bereich 5–100 nm typischerweise abschwächt; 6) warum PMBC-artige Protozellen aufgrund höherer Polarisierbarkeit stärkere Kandidaten für CL-vermittelte Stabilisierung sind als einfache Fettsäurevesikel; 7) wie man Simulationsergebnisse sauber an Team 2 übergibt.

3. Physikalischer Grundsatz: Was darf verglichen werden?

Das Team soll vier getrennte Modellmodi implementieren und vergleichen. Erstens: vdW-only als nicht-retardierter Nahbereichsgrenzfall. Zweitens: klassisches DLVO als elektrostatische Doppelschicht-Repulsion plus nicht-retardierte vdW-Anziehung plus kurzer Core-Term. Drittens: retardierte CL-Approximation als vdW-Grenzfall mit Retardierungsfaktor. Viertens: ein dokumentierter Ausblick auf die spätere Lifshitz-Berechnung mit Matsubara-Moden, TE/TM-Polarisationen und Schalenmodell.

Wichtig: Ein Modell F_total = F_el + F_vdW + F_CL ist in der Regel physikalisch problematisch, weil vdW bereits Teil des Lifshitz-Rahmens ist. Nur ein zusätzlicher, explizit als Hypothese deklarierter mesoskopischer CL-Term dürfte separat getestet werden.

Merksatz für das Seminar

DLVO beschreibt klassische Nahbereichs-Kolloidphysik mit stark gescreenter Elektrostatik. vdW ist der nicht-retardierte Nahbereich der Lifshitz-Theorie. Retardierte Casimir–Lifshitz-Kräfte bleiben in Salzlösungen über höhere Matsubara-Moden relevant, werden aber nicht automatisch stärker. Für stabile Protozellencluster zählt das Zusammenspiel aus Material, Abstand, Salz, Retardierung, Rauschen und Mehrkörpergeometrie.

4. Gemeinsamer Parameterraum

Alle Rechnungen sollen zunächst mit einem einheitlichen Referenzsystem durchgeführt werden. Danach folgen Sensitivitätsanalysen.

Parameter	Wert	Bedeutung
Protozellenradius	R = 500 nm	Referenzwert für alle Basiskurven
Effektiver Radius	R_eff = R/2 = 250 nm	Für zwei gleich große Kugeln in Derjaguin-/PFA-Näherung
Oberflächenabstand	L = 2–100 nm	L ist nicht der Mittelpunktabstand, sondern der Abstand der äußeren Membranoberflächen
Mittelpunktabstand	d = 2R + L	Wichtig für Simulationen mit Positionen
Temperatur	T = 298.15 K	Zimmertemperatur als Referenz
Ionenstärke	I = 50, 100, 200 mM	präbiotisch plausible Salzfenster / Ursuppe
Debye-Länge	ca. 1.36, 0.96, 0.68 nm	für 50, 100, 200 mM monovalentes Salz

Zone	Abstandsbereich	Interpretation
hard_contact	L < 2 nm	Core, Hydration, numerisch sehr empfindlich
dlvo_zone	2 ≤ L < 5 nm	Elektrostatische Repulsion kann noch relevant sein
near_gap	5 ≤ L < 10 nm	Übergang: Elektrostatik stark gedämpft, vdW/CL relevant
meso_gap	10 ≤ L < 30 nm	Retardierung und Materialantwort wichtig
far_gap	30 ≤ L < 100 nm	Einzelpaar-Kräfte schwach; Mehrkörper-/Geometrieeffekte nötig
separated	L ≥ 100 nm	Für Team 2 als getrennt behandeln

5. Modellstufe 1 – nicht-retardierte van-der-Waals-Anziehung

Diese Stufe ist die klassische Kurzdistanz-Baseline. Für zwei gleiche Kugeln im PFA-/Derjaguin-Limit wird die nicht-retardierte Hamaker-Anziehung verwendet:

U_vdW(L) = - A_H * R_eff / (6 L)
F_vdW(L) = - A_H * R_eff / (6 L^2)

Die Kraft ist negativ, wenn sie anziehend wirkt. Diese Form ist nur der Nahbereichsgrenzfall der Lifshitz-Theorie und sollte nicht als unabhängige Zusatzkraft neben einer vollständigen CL-Berechnung addiert werden.

6. Modellstufe 2 – klassisches DLVO-Modell

Das klassische DLVO-Modell besteht aus drei Teilen: elektrostatische Doppelschicht-Repulsion, nicht-retardierte vdW-Anziehung und kurzer Core-/Hydrationsterm zur Stabilisierung. Für die Seminarimplementierung genügt zunächst:

U_DLVO(L) = U_el(L) + U_vdW(L) + U_core(L)
U_el(L) = B exp(-kappa L)
U_core(L) = C (L_core/L)^12

Die elektrostatische Repulsion wird durch Salz stark Debye-abgeschirmt. Bei 100 mM ist lambda_D ungefähr 0.96 nm. Deshalb ist U_el oberhalb weniger Nanometer sehr klein. Die nicht-retardierte vdW-Komponente wird dagegen nicht einfach durch exp(-kappa L) abgeschirmt, sondern über die frequenzabhängige Materialantwort und den Lifshitz-Hamaker-Beitrag verändert.

7. Modellstufe 3 – retardierte Casimir–Lifshitz-Approximation

Retardierung bedeutet, dass elektromagnetische Fluktuationen nicht mehr quasistatisch gekoppelt sind. Mit zunehmendem Abstand koppeln schnelle Moden schlechter, und die effektive Anziehung wird gegenüber der nicht-retardierten vdW-Grenze typischerweise schwächer.

Für ein didaktisch handhabbares Modell wird die retardierte CL-Kraft zunächst als abgeschwächter vdW-Grenzfall modelliert:

F_CL,ret(L) = F_vdW(L) * r(L)
r(L) = 1 / (1 + L/L_r)^p

Empfohlene Startwerte: L_r = 20 nm, p = 2. Zusätzlich sollen L_r = 10, 20, 30 nm und p = 1, 2 getestet werden.

8. Modellstufe 4 – Ausblick: Matsubara, TE/TM und Elektrolyt

Für die spätere präzisere Theorie soll das Team dokumentieren, wie die Lifshitz-Kraft korrekt zerlegt würde:

F_CL(L) = F_0,long(L,kappa) + F_0,TM(L) + F_{n>=1}^{TE/TM}(L)

Der n=0-longitudinale Beitrag ist Debye-gescreent und vor allem im Bereich weniger Debye-Längen relevant. Die höheren Matsubara-Moden n>=1 sind für typische Salzlösungen deutlich weniger direkt durch die Ionenbewegung betroffen. Für genauere Rechnungen müssen TE- und TM-Polarisationen getrennt berücksichtigt werden. In diesem Aufgabenblatt wird dies nicht vollständig numerisch verlangt, aber als theoretischer Entwicklungsweg sauber beschrieben.

Materialmodell	A_H / A_eff-Sweep	Interpretation
Fettsäurevesikel	0.2, 0.5, 1.0 × 10^-21 J	dünne, weichere, schwächer polarisierbare Schale
PMBCs	1, 3, 5, 10 × 10^-21 J	protein-/polymerreicher, dicker, polariserbarer
Referenzvergleich	5 × 10^-21 J	nur für PMBC-artige obere Abschätzung verwenden

9. Schritt-für-Schritt-Arbeitsplan für die Modellierung

Schritt 1: Einheiten konsequent festlegen

Alle Längen in Meter, Kräfte in Newton intern und pN in Tabellen, Energien in Joule intern und U/kBT in Ausgaben. Fehler in Einheiten sind die häufigste Quelle falscher Ergebnisse.

Schritt 2: L-Gitter erzeugen

Verwendet ein logarithmisches Gitter von 2 bis 100 nm und zusätzlich die Stützstellen 2, 3, 5, 10, 20, 30, 50, 100 nm.

Schritt 3: vdW-only implementieren

Berechnet U_vdW(L), F_vdW(L), U/kBT und F in pN für Fettsäurevesikel und PMBCs.

Schritt 4: DLVO implementieren

Fügt U_el und U_core hinzu. Testet I = 50, 100, 200 mM. Prüft, ab welchem L die elektrostatische Repulsion praktisch verschwindet.

Schritt 5: CL-retarded implementieren

Implementiert r(L). Vergleicht F_vdW mit F_CL,ret und zeigt, wie stark Retardierung bei 5, 10, 20, 30 nm wirkt.

Schritt 6: Kraft- und Potentialplots erstellen

Plot 1: Kraft in pN gegen L. Plot 2: U/kBT gegen L. Plot 3: Retardierungsfaktor gegen L. Plot 4: DLVO vs CL-retarded für Fettsäure und PMBC.

Schritt 7: Paarmodell simulieren

Simuliert L(t) mit Brownian Dynamics und messt Kontaktwahrscheinlichkeit, Escape Time, Rückkehrwahrscheinlichkeit und Verweildauer in Abstandszonen.

Schritt 8: Mehrteilchenmodell

Simuliert N=10–50 Protozellen in 2D. Exportiert Positionen, Paarabstände, Kontaktzonen und Kontaktgraphen.

Schritt 9: Sensitivitätsanalyse

Sweeps über A_H, Salz, Rauschen, L_r, Radius und Core-Parameter. Haltet Seeds konstant, damit Teams vergleichen können.

Schritt 10: Übergabe an Team 2

Exportiert saubere CSV-Dateien mit run_id, seed, t, particle_id, x, y sowie Paarabständen und Kontaktzonen.

10. Erwartete qualitative Ergebnisse

Abstand	DLVO-Erwartung	CL/vdW-Erwartung
2–5 nm	Elektrostatische DLVO-Repulsion kann netto dominieren; Core wichtig	vdW/CL stark, aber Nahbereich; sehr empfindlich
5–10 nm	Elektrostatik stark reduziert; Übergangsbereich	PMBCs können noch relevante Anziehung zeigen
10–30 nm	DLVO-Elektrostatik praktisch weg; vdW/retardierte CL schwach bis moderat	für PMBCs plausibel, für Fettsäure eher grenzwertig
30–100 nm	klassische Einzelpaar-DLVO sehr schwach	Einzelpaar-CL meist schwach; Mehrkörper-/Gap-Effekte nötig

Strenge Interpretationsregel

Team 1 darf am Ende nicht behaupten: „CL erklärt Clusterbildung von 2–100 nm vollständig.“ Sauberer ist: „Standard-Lifshitz-/CL-Kopplung kann im Bereich 2–30 nm plausibel zur Stabilisierung beitragen, besonders für PMBC-artige Protozellen. Für 30–100 nm sind Einzelpaar-Kräfte wahrscheinlich schwach; robuste Clusterbildung erfordert Mehrkörper-, Geometrie- oder Gap-Effekte.“

11. Metriken

Metrik	Definition	Nutzen
Kraftstärke	F(L) in pN	Vergleich der Modellbeiträge
Potentialtiefe	U(L)/kBT	thermische Relevanz
Kontaktwahrscheinlichkeit	P(L < L_contact)	Häufigkeit physikalischer Nähe
Dwell Time	zusammenhängende Zeit in Kontaktzone	Persistenz
Escape Time	Zeit bis Verlassen einer Zone	Stabilität
Return Probability	P(Rückkehr in Zone innerhalb T)	Attractor-Hinweis
Largest Component	größte zusammenhängende Komponente	Clusterbildung
Mean Degree	mittlerer Kontaktgraph-Grad	Netzwerkdichte

12. Vollständig formatierter Code-Anhang

Der folgende Code ist als Startpunkt gedacht. Er ist bewusst modular gehalten und soll von Team 1 erweitert, getestet und dokumentiert werden.

```
# src/constants.py
import numpy as np

kB = 1.380649e-23        # J/K
T_REF = 298.15           # K
NM = 1e-9
PN = 1e-12

DEBYE_LENGTHS = {
   50: 1.36e-9,
   100: 0.96e-9,
   200: 0.68e-9,
}

def kBT(T=T_REF):
   return kB * T


# src/potentials.py
import numpy as np
from constants import kB, T_REF

class ProtocellForces:
   def __init__(
      self,
      R=500e-9,
      A_H=1e-21,
      lambda_D=0.96e-9,
      B_kBT=20.0,
      T=T_REF,
      L_core=0.5e-9,
      C_core_kBT=0.5,
      L_ret=20e-9,
      p_ret=2,
      L_min=0.2e-9,
   ):
      self.R = R
      self.R_eff = R / 2.0
      self.A_H = A_H
      self.lambda_D = lambda_D
      self.kappa = 1.0 / lambda_D
      self.T = T
      self.B = B_kBT * kB * T
      self.L_core = L_core
      self.C_core = C_core_kBT * kB * T
```

```
    self.L_ret = L_ret
    self.p_ret = p_ret
    self.L_min = L_min

def clip(self, L):
    return np.maximum(L, self.L_min)

def U_vdw(self, L):
    L = self.clip(L)
    return -self.A_H * self.R_eff / (6.0 * L)

def F_vdw(self, L):
    L = self.clip(L)
    return -self.A_H * self.R_eff / (6.0 * L**2)

def U_el(self, L):
    L = self.clip(L)
    return self.B * np.exp(-self.kappa * L)

def F_el(self, L):
    L = self.clip(L)
    return self.kappa * self.U_el(L)

def U_core(self, L):
    L = self.clip(L)
    return self.C_core * (self.L_core / L)**12

def F_core(self, L):
    L = self.clip(L)
    return 12.0 * self.C_core * self.L_core**12 / L**13

def U_dlvo(self, L):
    return self.U_el(L) + self.U_vdw(L) + self.U_core(L)

def F_dlvo(self, L):
    return self.F_el(L) + self.F_vdw(L) + self.F_core(L)

def retardation_factor(self, L):
    L = self.clip(L)
    return 1.0 / (1.0 + L / self.L_ret)**self.p_ret

def F_cl_retarded(self, L):
    return self.F_vdw(L) * self.retardation_factor(L)

def U_cl_retarded_numeric(self, L_grid):
    """
    Numerische Integration aus F = -dU/dL.
    Für didaktische Zwecke: U(L_max)=0 setzen und rückwärts integrieren.
```

```
        """
        L = np.asarray(L_grid)
        F = self.F_cl_retarded(L)
        U = np.zeros_like(L)
        for i in range(len(L)-2, -1, -1):
            dL = L[i+1] - L[i]
            U[i] = U[i+1] - 0.5 * (F[i+1] + F[i]) * dL
        return U


# src/force_table.py
import pandas as pd
from constants import kB, T_REF, PN
from potentials import ProtocellForces

def force_table(model, L_nm_values=(2,3,5,10,20,30,50,100)):
    rows = []
    for L_nm in L_nm_values:
        L = L_nm * 1e-9
        rows.append({
            "L_nm": L_nm,
            "F_vdw_pN": model.F_vdw(L) / PN,
            "F_el_pN": model.F_el(L) / PN,
            "F_dlvo_pN": model.F_dlvo(L) / PN,
            "F_cl_ret_pN": model.F_cl_retarded(L) / PN,
            "U_vdw_kBT": model.U_vdw(L) / (kB * T_REF),
            "U_dlvo_kBT": model.U_dlvo(L) / (kB * T_REF),
            "retardation_factor": model.retardation_factor(L),
        })
    return pd.DataFrame(rows)

if __name__ == "__main__":
    fatty = ProtocellForces(A_H=0.5e-21)
    pmbc = ProtocellForces(A_H=5e-21)
    print("Fatty-acid vesicle")
    print(force_table(fatty))
    print("PMBC")
    print(force_table(pmbc))


# src/pair_dynamics.py
import numpy as np

class PairBrownian1D:
    def __init__(self, force_model, L0=20e-9, dt=1e-4, mobility=1e8,
                 diffusion=1e-16, L_min=0.5e-9, L_max=200e-9, seed=0):
        self.force_model = force_model
        self.L = L0
        self.dt = dt
```

```
        self.mobility = mobility
        self.diffusion = diffusion
        self.L_min = L_min
        self.L_max = L_max
        self.rng = np.random.default_rng(seed)

    def step(self, mode="dlvo"):
        if mode == "dlvo":
            F = self.force_model.F_dlvo(self.L)
        elif mode == "vdw":
            F = self.force_model.F_vdw(self.L)
        elif mode == "cl_retarded":
            F = self.force_model.F_cl_retarded(self.L)
        else:
            raise ValueError("unknown mode")

        drift = self.mobility * F * self.dt
        noise = np.sqrt(2 * self.diffusion * self.dt) * self.rng.normal()
        self.L = self.L + drift + noise
        self.L = min(max(self.L, self.L_min), self.L_max)
        return self.L

    def run(self, steps=10000, mode="dlvo"):
        traj = np.zeros(steps)
        for t in range(steps):
            traj[t] = self.step(mode=mode)
        return traj

def contact_zone(L):
    if L < 2e-9:
        return "hard_contact"
    if L < 5e-9:
        return "dlvo_zone"
    if L < 10e-9:
        return "near_gap"
    if L < 30e-9:
        return "meso_gap"
    if L < 100e-9:
        return "far_gap"
    return "separated"


# src/brownian_cluster.py
import numpy as np
from pair_dynamics import contact_zone

class BrownianClusterSim:
    def __init__(self, n=20, R=500e-9, box=20e-6, dt=1e-4,
                 mobility=1e8, diffusion=1e-16, seed=0):
```

```
        self.rng = np.random.default_rng(seed)
        self.n = n
        self.R = R
        self.box = box
        self.dt = dt
        self.mobility = mobility
        self.diffusion = diffusion
        self.x = self.rng.uniform(0, box, size=(n, 2))

    def minimum_image(self, delta):
        return delta - self.box * np.round(delta / self.box)

    def step(self, force_model, mode="dlvo"):
        forces = np.zeros_like(self.x)
        for i in range(self.n):
            for j in range(i + 1, self.n):
                delta = self.minimum_image(self.x[j] - self.x[i])
                dist = np.linalg.norm(delta) + 1e-30
                L = max(dist - 2 * self.R, 0.2e-9)
                direction = delta / dist
                if mode == "dlvo":
                    F = force_model.F_dlvo(L)
                elif mode == "vdw":
                    F = force_model.F_vdw(L)
                elif mode == "cl_retarded":
                    F = force_model.F_cl_retarded(L)
                else:
                    raise ValueError("unknown mode")
                forces[i] += F * direction
                forces[j] -= F * direction
        noise = np.sqrt(2 * self.diffusion * self.dt) * self.rng.normal(size=self.x.shape)
        self.x += self.mobility * forces * self.dt + noise
        self.x = np.mod(self.x, self.box)
        return self.x

    def pair_distance_table(self):
        rows = []
        for i in range(self.n):
            for j in range(i + 1, self.n):
                delta = self.minimum_image(self.x[j] - self.x[i])
                dist = np.linalg.norm(delta)
                L = max(dist - 2 * self.R, 0.0)
                rows.append((i, j, L, contact_zone(L)))
        return rows


# src/metrics.py
import numpy as np
import networkx as nx
```

```
CONTACT_ZONES = {"hard_contact", "dlvo_zone", "near_gap", "meso_gap"}

def contact_graph(pair_rows, n, accepted_zones=CONTACT_ZONES):
    g = nx.Graph()
    g.add_nodes_from(range(n))
    for i, j, L, zone in pair_rows:
        if zone in accepted_zones:
            g.add_edge(i, j, L=L, zone=zone)
    return g

def cluster_metrics(pair_rows, n):
    g = contact_graph(pair_rows, n)
    comps = [len(c) for c in nx.connected_components(g)]
    largest = max(comps) if comps else 1
    mean_degree = float(np.mean([d for _, d in g.degree()])) if n else 0.0
    return {
        "largest_cluster": largest,
        "n_components": len(comps),
        "mean_degree": mean_degree,
        "n_edges": g.number_of_edges(),
    }

def dwell_times(zone_sequence, target_zone):
    out = []
    current = 0
    for z in zone_sequence:
        if z == target_zone:
            current += 1
        else:
            if current > 0:
                out.append(current)
                current = 0
    if current > 0:
        out.append(current)
    return out
```

13. Empfohlene Projektstruktur

```
team1_forces/
├── README.md
├── requirements.txt
├── src/
│   ├── constants.py
│   ├── potentials.py
│   ├── force_table.py
│   ├── pair_dynamics.py
```

```
│   ├── brownian_cluster.py
│   ├── metrics.py
│   └── plots.py
├── notebooks/
│   ├── 01_force_curves.ipynb
│   ├── 02_pair_dynamics.ipynb
│   ├── 03_cluster_simulation.ipynb
│   └── 04_parameter_sweep.ipynb
├── data/
│   ├── raw/
│   └── processed/
├── figures/
└── report/
```

14. Pflichtvisualisierungen

Kraftkurven F(L) in pN für vdW-only, elektrostatische Repulsion, DLVO und CL-retarded.

Potentialkurven U(L)/kBT für mindestens Fettsäurevesikel und PMBCs.

Retardierungsfaktor r(L) für L_r = 10, 20, 30 nm und p = 1, 2.

Paartrajektorien L(t) für typische Läufe in DLVO und CL-retarded.

Histogramme der Verweildauern in dlvo_zone, near_gap und meso_gap.

Clustergrößenverteilungen für 10–50 Protozellen.

Parameter-Sweep-Heatmap: A_H gegen Rauschen oder Salz, Farbe = mittlere Clustergröße / Persistenz.

15. Wochenplan

Woche	Schwerpunkt	Ergebnis
1	Theorie und Einheiten	Parameterraum, Einheiten, erster vdW-Code
2	DLVO und Retardierung	Kraft-/Potentialkurven für vdW, DLVO, CL-retarded
3	Paarmodell	Brownian-Pair-Dynamik, Kontaktzonen, Escape/Return-Metriken
4	Mehrteilchenmodell	2D-Simulation, Kontaktgraphen, Clustergrößen
5	Sensitivitätsanalyse I	A_H, Salz, L_r, p, Radius
6	Sensitivitätsanalyse II	Rauschen, Core, Kontaktdefinition, Negativkontrollen
7	Datenübergabe und Validierung	CSV-Export für Team 2, Reproduzierbarkeit, Seeds
8	Bericht und Präsentation	Interpretation, Grenzen, finaler Modellvergleich

16. Bewertungsrubrik

Kriterium	Gewicht	Anforderung
Physikalische Sauberkeit	25%	vdW/DLVO/CL korrekt abgegrenzt; keine Doppelzählung; klare Einheiten und Parameter
Implementierung	20%	modularer, reproduzierbarer Python-Code; stabile Simulation; dokumentierte Seeds
Analyse und Metriken	20%	Kraft-/Potentialkurven, Paarmetriken, Clustermetriken, Sensitivitätsanalyse
Datenübergabe	15%	saubere CSV-Formate, Kontaktzonen, Metadaten, Team-2-Kompatibilität
Interpretation	15%	keine Überbehauptungen; klare Aussagen zu 2–10, 10–30 und 30–100 nm
Darstellung	5%	lesbare Abbildungen, Tabellen, Berichtstruktur

17. Übergabeformat an Team 2

Datei	Spalten	Nutzen
positions.csv	run_id, seed, t, particle_id, x, y, model_type, A_H, lambda_D, noise, T	Basis für Makrozustände
pairs.csv	run_id, seed, t, i, j, L_nm, contact_zone, force_mode	Abstandszonen und Kontaktgraphen
cluster_metrics.csv	run_id, seed, t, largest_cluster, mean_degree, n_edges, n_components	direkte Team-2-Metriken
parameters.csv	run_id, model_type, R_nm, A_H, salt_mM, L_ret_nm, p_ret, dt, mobility, diffusion	Reproduzierbarkeit

18. Typische Fehler und wie sie vermieden werden

Fehler 1: vdW und CL addieren

Vermeidung: vdW ist der nicht-retardierte Grenzfall der Lifshitz-Theorie. CL-retarded ersetzt oder modifiziert vdW, es wird nicht blind addiert.

Fehler 2: L mit Mittelpunktabstand verwechseln

Vermeidung: L ist Oberflächenabstand; Mittelpunktabstand ist d = 2R + L.

Fehler 3: elektrostatische Abschirmung auf vdW übertragen

Vermeidung: Debye-Screening betrifft statische/longitudinale Elektrostatik stark, nicht einfach alle Fluktuationsmoden.

Fehler 4: Retardierung als Verstärkung interpretieren

Vermeidung: Retardierung schwächt die nicht-retardierte vdW-Grenze meist ab.

Fehler 5: 30–100 nm überbehaupten

Vermeidung: Einzelpaar-Kräfte sind dort schwach; nur Mehrkörper-/Geometrieeffekte als Hypothese formulieren.

Fehler 6: numerische Explosionen bei kleinen L

Vermeidung: L_min, Core-Term, kleine Zeitschritte und Kraftbegrenzung testen.

19. Abschlussauftrag an Team 1

Entwickeln Sie ein transparentes, reproduzierbares physikalisches Minimalmodell für die Wechselwirkung präbiotischer Protozellen mit R = 500 nm im Bereich L = 2–100 nm. Implementieren Sie getrennt vdW-only, klassisches DLVO und eine retardierte Casimir–Lifshitz-Approximation. Vergleichen Sie Kräfte, Potentiale, Brownian-Pair-Dynamik und Mehrteilchencluster unter Variation von Materialparametern, Salz, Rauschen und Retardierung. Dokumentieren Sie exakt, in welchem Abstandsbereich welches Modell plausibel zur Kopplung beitragen kann, und exportieren Sie robuste Mikrodaten für Team 2.

20. Literatur- und Theoriehinweise für Team 1

Israelachvili, Intermolecular and Surface Forces: klassische DLVO-, vdW- und Hydration-Grundlagen.

Casimir und Lifshitz: Ursprung und Verallgemeinerung fluktuationsinduzierter Kräfte.

Nunes et al. / Spreng et al. zu Casimir-Interaktionen zwischen Kugeln in Elektrolyten: n=0-Beitrag, Debye-Hückel, Kugel-Kugel-Geometrie.

Schoger et al. zu universellen Casimir-Interaktionen dielektrischer Kugeln in Salzwasser: niedrigfrequente transversale thermische Fluktuationen.

Seyedzahedi und Moradian zu Elektrolyten, Schichtmedien, TE/TM und Hamaker-Koeffizienten.